1.Index.cshtml code

```
@{
   ViewData["Title"] = "Home Page";
}


<div class="text-center">
   <h1 class="display-4">Product Management</h1>
</div>


<div class="container mt-4">
   <button class="btn btn-primary mb-3" data-bs-toggle="modal" data-bs-
target="#productModel" onclick="openAddModal()">Add Product</button>
   <table class="table table-striped">
      <thead>
         <tr>
            <th>Product Name</th>
            <th>Address</th>
            <th>Country</th>
            <th>State</th>
            <th>City</th>
            <th>Production Document</th>
            <th>Actions</th>
         </tr>
      </thead>
      <tbody id="productTableBody"></tbody>
   </table>
</div>
```

```
<!-- Modal for Adding/Editing Product -->
<div class="modal fade" id="productModel" tabindex="-1" aria-labelledby="productModelLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <form id="productForm" enctype="multipart/form-data">
        <div class="modal-header">
          <h5 class="modal-title" id="productModelLabel">Add/Edit Product</h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
        </div>
        <div class="modal-body">
          <input type="hidden" id="productId" name="Id" />
          <div class="mb-3">
            <label for="productName" class="form-label">Product Name</label>
            <input type="text" class="form-control" id="productName" name="ProductName" required />
          </div>
          <div class="mb-3">
            <label for="address" class="form-label">Address</label>
            <input type="text" class="form-control" id="address" name="Address" required />
          </div>
          <div class="mb-3">
            <label for="country" class="form-label">Country</label>
            <select class="form-select" id="country" name="Country" onchange="loadStates()"></select>
          </div>
          <div class="mb-3">
```

```
                <label for="state" class="form-label">State</label>
                <select class="form-select" id="state" name="State"
onchange="loadCities()"></select>
            </div>
            <div class="mb-3">
                <label for="city" class="form-label">City</label>
                <select class="form-select" id="city" name="City"></select>
            </div>
            <div class="mb-3">
                <label for="productionDocumentation" class="form-label">Production
Document</label>
                <input type="file" class="form-control" id="productionDocumentation"
name="ProductionDocumentation" />
            </div>
          </div>
          <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Close</button>
            <button type="submit" class="btn btn-primary"
id="saveProductBtn">Save</button>
          </div>
        </form>
      </div>
    </div>
</div>

<div id="notification"></div>

@section Scripts {
```

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
<script>
  const countries = {
    "USA": ["California", "Texas"],
    "Canada": ["Ontario", "Quebec"],
    "India": ["Maharashtra", "Kerala"],
    "Australia": ["Sydney"]
  };

  const cities = {
    "California": ["Los Angeles", "San Francisco", "San Diego"],
    "Texas": ["Houston", "Dallas", "Austin"],
    "Ontario": ["Toronto", "Ottawa", "Hamilton"],
    "Quebec": ["Montreal", "Quebec City", "Laval"],
    "Maharashtra": ["Mumbai", "Pune", "Nagpur"],
    "Kerala": ["Kochi", "Thiruvananthapuram", "Kozhikode"],
    "Sydney": ["Sydney"]
  };

  $(document).ready(() => {
    loadCountries();
    loadProducts();
  });

  function loadCountries() {
    let countrySelect = $('#country');
    countrySelect.empty();
    countrySelect.append('<option value="">Select Country</option>');
```

```
        for (let country in countries) {
            countrySelect.append(` <option value="${country}">${country}</option>` );
        }
    }

    function loadStates(selectedState = "") {
        let country = $('#country').val();
        let stateSelect = $('#state');
        stateSelect.empty();
        stateSelect.append('<option value="">Select State</option>');
        if (countries[country]) {
            countries[country].forEach(state => {
                stateSelect.append(` <option value="${state}" ${state == selectedState ? 'selected' : ''}>${state}</option>` );
            });
        }
        loadCities();
    }

    function loadCities(selectedCity = "") {
        let state = $('#state').val();
        let citySelect = $('#city');
        citySelect.empty();
        citySelect.append('<option value="">Select City</option>');
        if (cities[state]) {
            cities[state].forEach(city => {
                citySelect.append(` <option value="${city}" ${city === selectedCity ? 'selected' : ''}>${city}</option>` );
            });
```

```
    }
}

function openEditModal(productId) {
    $.ajax({
        url: ` /api/Products/${productId}` ,
        type: "GET",
        success: function (product) {
            $('#productId').val(product.id);
            $('#productName').val(product.productName);
            $('#address').val(product.address);
            $('#country').val(product.country);
            loadStates(product.state);
            loadCities(product.city);
            $('#productionDocumentation').val('');
            $('#productModelLabel').text("Edit Product");
            $('#productModel').modal('show');
        },
        error: function () {
            showNotification("Failed to load details.", "danger");
        }
    });
}

function openAddModal() {
    $('#productForm').trigger("reset");
    $('#productId').val('');
    $('#productModelLabel').text("Add Product");
```

```
        $('#saveProductBtn').text("Save");
        $('#productModel').modal('show');
    }

    function showNotification(message, type) {
        let notification = `<div class="alert alert-${type} alert-dismissible fade show" role="alert">
                            ${message}
                            <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
                            </div>`;
        $("#notification").html(notification);
    }

    function confirmDelete(productId) {
        if (!confirm("Are you sure you want to delete this product?")) return;
        $.ajax({
            url: `/api/Products/${productId}`,
            type: "DELETE",
            success: function () {
                loadProducts();
                showNotification("Product deleted successfully.", "success");
            },
            error: function () {
                showNotification("Failed to delete product.", "danger");
            }
        });
    }
```

```
    $('#productForm').on('submit', function (e) {
        e.preventDefault();
        const productId = $('#productId').val();
        const productData = new FormData(this);

        const ajaxOptions = {
            data: productData,
            processData: false,
            contentType: false,
            success: function () {
                $('#productModel').modal('hide');
                loadProducts();
                showNotification(productId ? "Product updated successfully." : "Product
added successfully", "success");
            },
            error: function (jqXHR) {
                const errorResponse = jqXHR.responseJSON;
                showNotification(errorResponse?.title || "Failed to process request.",
"danger");
            }
        };

        if (productId) {
            ajaxOptions.url = `/api/Products/${productId}`;
            ajaxOptions.type = "PUT";
        } else {
            ajaxOptions.url = "/api/Products";
            ajaxOptions.type = "POST";
        }
```

```
    $.ajax(ajaxOptions);
  });


  function loadProducts() {
    $.get("/api/Products", function (data) {
      let rows = "";
      data.forEach(product => {
        rows += `<tr>
          <td>${product.productName}</td>
          <td>${product.address}</td>
          <td>${product.country}</td>
          <td>${product.state}</td>
          <td>${product.city}</td>
          <td>
            ${product.productionDocumentationPath
              ? `<a href="${product.productionDocumentationPath}" class="btn btn-info btn-sm" target="_blank" download>Download</a>`
              : ""}
          </td>
          <td>
            <button class="btn btn-warning btn-sm" onclick="openEditModal(${product.id})">Edit</button>
            <button class="btn btn-danger btn-sm" onclick="confirmDelete(${product.id})">Delete</button>
          </td>
        </tr>`;
      });
      $("#productTableBody").html(rows);
    });
```

```
    }
  </script>
}
```

2.ProductsController.cs

```
using Microsoft.AspNetCore.Mvc;

[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
  private readonly DAL _dal;
  private readonly IWebHostEnvironment _env;

  public ProductsController(DAL dal, IWebHostEnvironment env)
  {
    _dal = dal;
    _env = env;
  }

  [HttpGet]
  public IActionResult GetAll()
  {
    return Ok(_dal.GetAll());
  }

  [HttpGet("{id}")]
  public IActionResult Get(int id)
```

```
  {
    var product = _dal.Get(id);
    if (product == null) return NotFound();
    return Ok(product);
  }

  [HttpPost]
  public async Task<IActionResult> Post([FromForm] Products product)
  {
    if (!ModelState.IsValid)
      return BadRequest(ModelState);

    if (product.ProductionDocumentation != null)
    {
      var fileName =
$"{Guid.NewGuid()}_{product.ProductionDocumentation.FileName}";
      var filePath = Path.Combine(_env.WebRootPath, "uploads", fileName);
      Directory.CreateDirectory(Path.GetDirectoryName(filePath)!);
      using (var stream = new FileStream(filePath, FileMode.Create))
      {
        await product.ProductionDocumentation.CopyToAsync(stream);
      }
      product.ProductionDocumentationPath = $"/uploads/{fileName}";
    }

    _dal.Add(product);
    return Ok();
  }
```

```
[HttpPut("{id}")]
public async Task<IActionResult> Put(int id, [FromForm] Products product)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);

    var existing = _dal.Get(id);
    if (existing == null) return NotFound();

    // Delete old file if new one is uploaded
    if (product.ProductionDocumentation != null)
    {
        if (!string.IsNullOrEmpty(existing.ProductionDocumentationPath))
        {
            var oldFilePath = Path.Combine(_env.WebRootPath,
existing.ProductionDocumentationPath.TrimStart('/'));
            if (System.IO.File.Exists(oldFilePath))
                System.IO.File.Delete(oldFilePath);
        }

        var fileName =
$"{Guid.NewGuid()}_{product.ProductionDocumentation.FileName}";
        var filePath = Path.Combine(_env.WebRootPath, "uploads", fileName);
        Directory.CreateDirectory(Path.GetDirectoryName(filePath)!);
        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await product.ProductionDocumentation.CopyToAsync(stream);
        }
```

```
            product.ProductionDocumentationPath = $"/uploads/{fileName}";
        }
        else
        {
            product.ProductionDocumentationPath =
existing.ProductionDocumentationPath;
        }

        product.Id = id;
        _dal.Update(product);
        return Ok();
    }

    [HttpDelete("{id}")]
    public IActionResult Delete(int id)
    {
        var product = _dal.Get(id);
        if (product == null) return NotFound();

        // Delete associated file
        if (!string.IsNullOrEmpty(product.ProductionDocumentationPath))
        {
            var filePath = Path.Combine(_env.WebRootPath,
product.ProductionDocumentationPath.TrimStart('/'));
            if (System.IO.File.Exists(filePath))
                System.IO.File.Delete(filePath);
        }

        _dal.Delete(id);
```

```
        return Ok();
    }
}
```

3.Program.cs

```
using DemoFullStackApplication.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<DAL>();
builder.Services.AddControllersWithViews();
builder.Services.AddRazorPages();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapRazorPages();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

```
app.MapControllers();

app.Run();
```

4.DAL.cs

```
using System.Collections.Generic;
using System.Linq;

public class DAL
{
   private static List<Products> _products = new List<Products>();
   private static int _nextId = 1;

   public List<Products> GetAll() => _products;

   public Products? Get(int id) => _products.FirstOrDefault(p => p.Id == id);

   public void Add(Products product)
   {
      product.Id = _nextId++;
      _products.Add(product);
   }

   public void Update(Products product)
   {
      var existing = Get(product.Id);
      if (existing != null)
```

```
    {
        existing.ProductName = product.ProductName;
        existing.Address = product.Address;
        existing.Country = product.Country;
        existing.State = product.State;
        existing.City = product.City;
        existing.ProductionDocumentationPath =
product.ProductionDocumentationPath;
    }
  }

  public void Delete(int id)
  {
    var product = Get(id);
    if (product != null)
      _products.Remove(product);
  }
}
```

5.Products.cs

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Http;

public class Products
{
  public int Id { get; set; }

  [Required]
  public string? ProductName { get; set; }
```

```
    [Required]
    public string? Address { get; set; }

    [Required]
    public string? Country { get; set; }

    [Required]
    public string? State { get; set; }

    [Required]
    public string? City { get; set; }

    // Not mapped to DB, used for upload
    public IFormFile? ProductionDocumentation { get; set; }

    // Path or name of uploaded file
    public string? ProductionDocumentationPath { get; set; }
}
```